



CSC4202

Landslide Emergency Supply Delivery Optimization

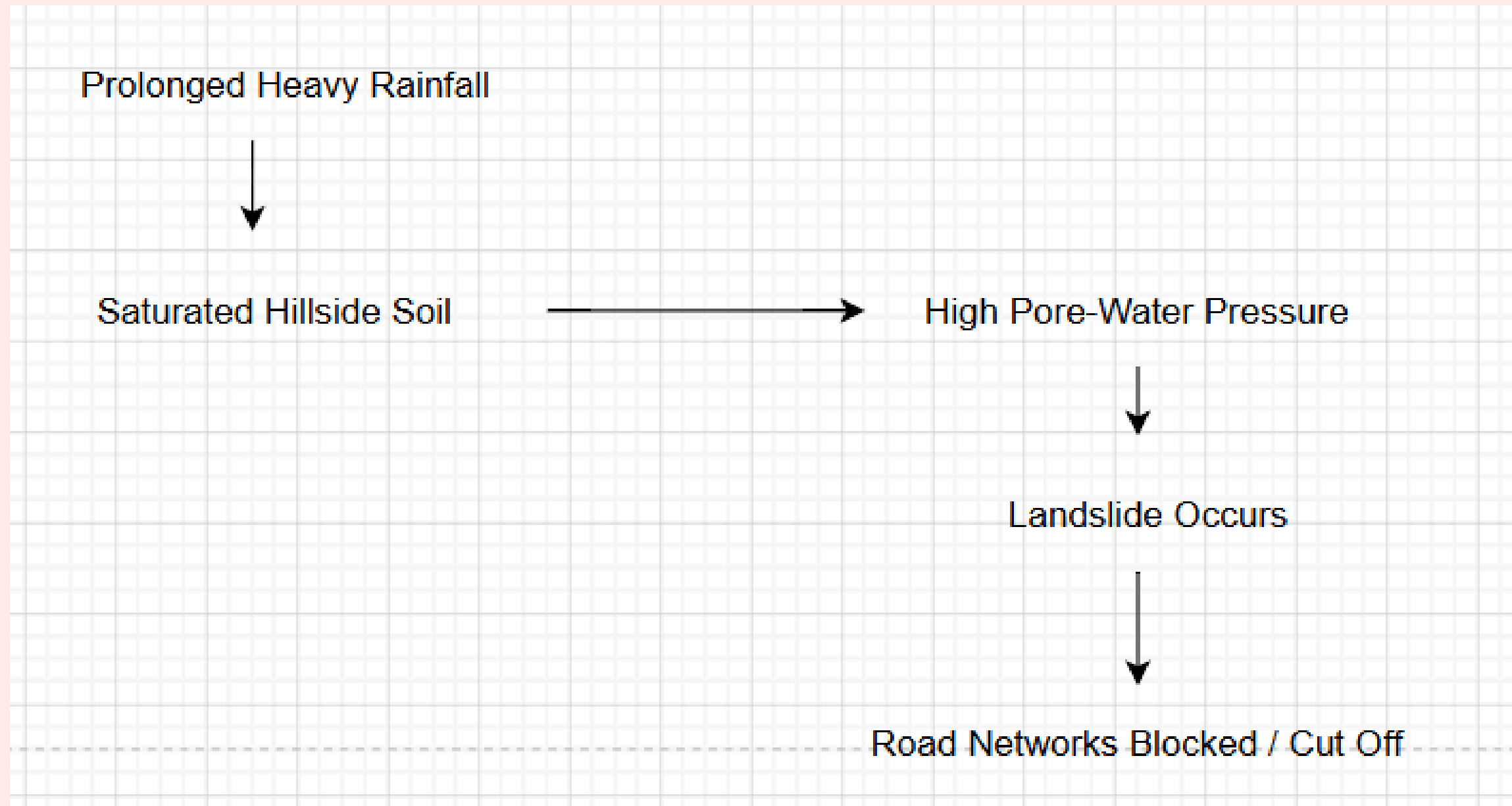
- RASYIDAH LIANA BINTI ABD HALIM 222239
- NUR SOFEA BINTI MUHALIS 224704
- NURUL SYAZANA BINTI ZAIDI 225552
- NURUL AINI BINTI ISMAIL 225068
- ALYA MAISARAH HANNANI BINTI AHMAD SUKRI 226080



Algorithmic Optimization for Emergency Relief Routing Under Dynamic Landslide Constraints

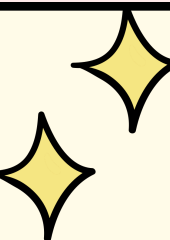
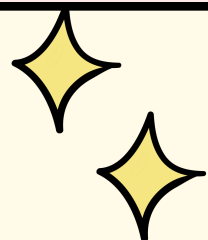
Case Study: Bentong District, Pahang (Northeast
Monsoon Logistics)

Environmental & Geological Drivers



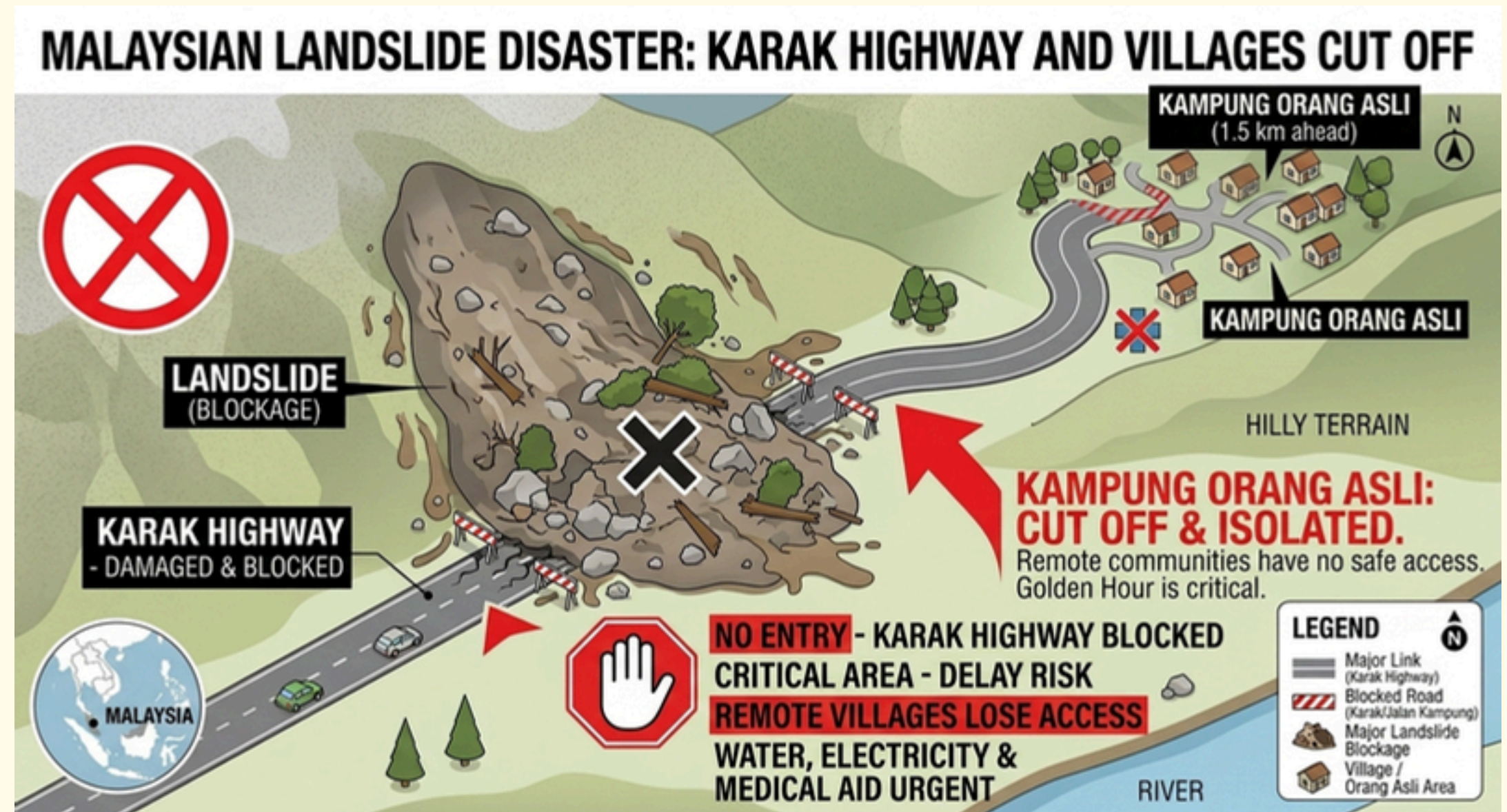
- Topographical Vulnerability: Foothills of Titiwangsa Range; steep slopes & heavily weathered granitic soil.
- Climatic Trigger: Northeast Monsoon (Nov–Mar) delivering 150 mm to 300 mm of rainfall within a 24-hour cycle.
- Geotechnical Failure: Pore-water pressure spike → Shear strength reduction → Slope failure.

Causal Mechanism Flowchart



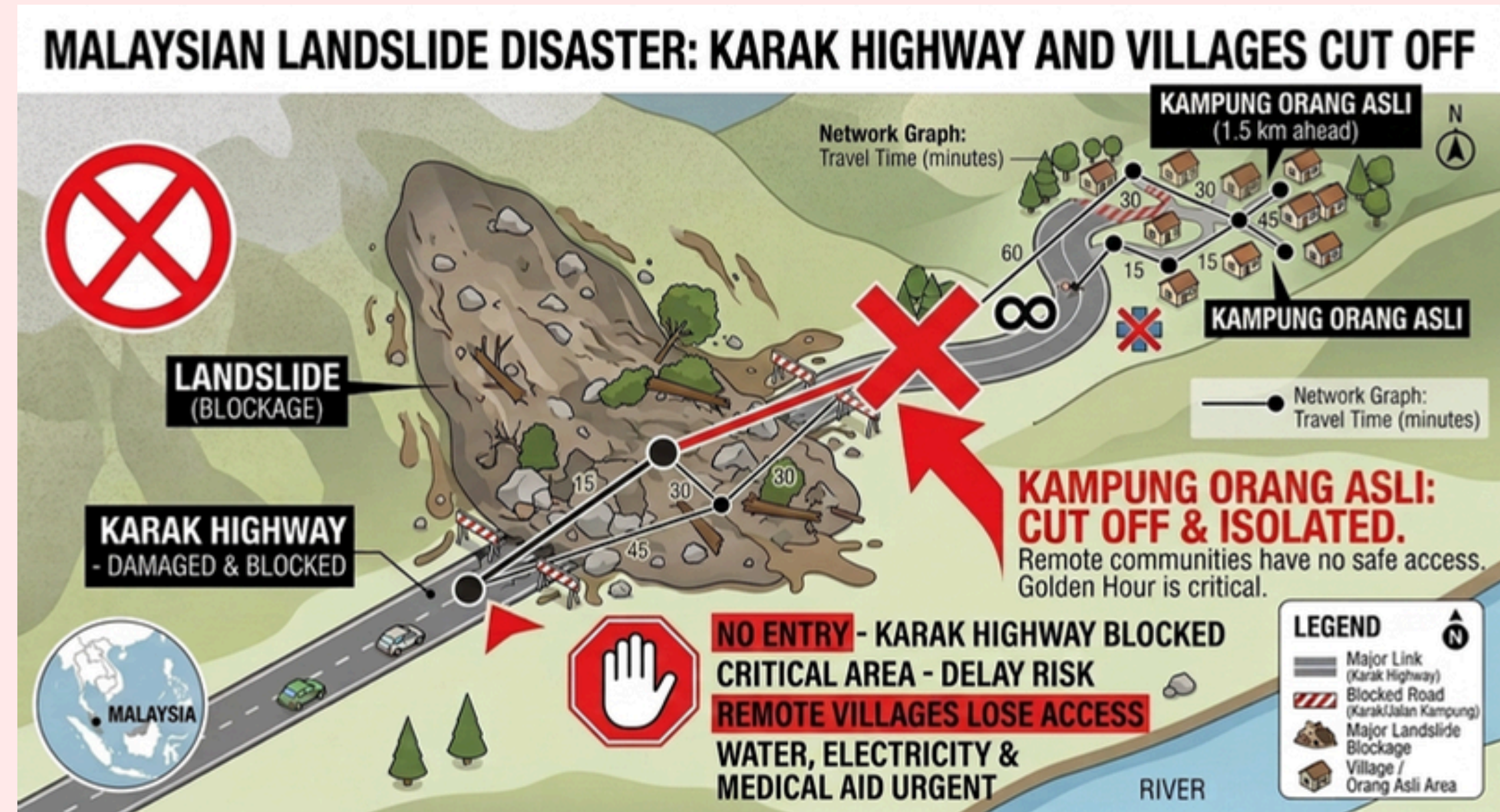
Impact of the Landslides

- Roads Blocked: Major links like the Karak Highway and small village roads (Jalan Kampung) are completely covered in mud and debris.
- Communities Isolated: Remote villages and Orang Asli areas lose access to water, electricity, and medical care instantly.
- The "Golden Hour": In an emergency, the first few hours are critical. Any delay in delivering supplies directly risks lives.



Turning the Problem Into Math

- Mathematical Modeling: Road network defined as a disrupted graph $G = (V, E)$.
- Dynamic Weight Alteration: Debris blockages render impacted edge weights as infinite ($w(e) = \infty$).
- System Vulnerabilities: Manual dispatch causes suboptimal path choices, forced backtracking, acute resource depletion, and compounded casualties.



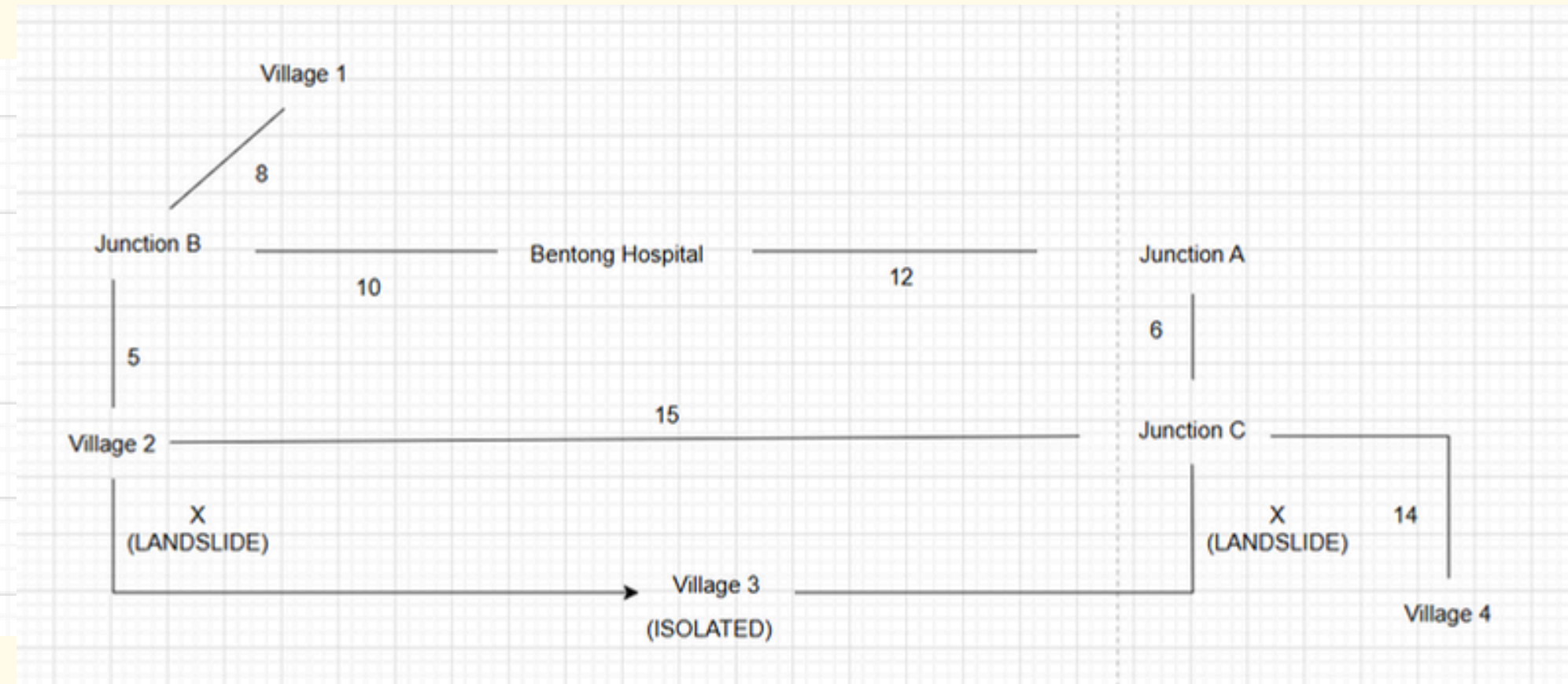
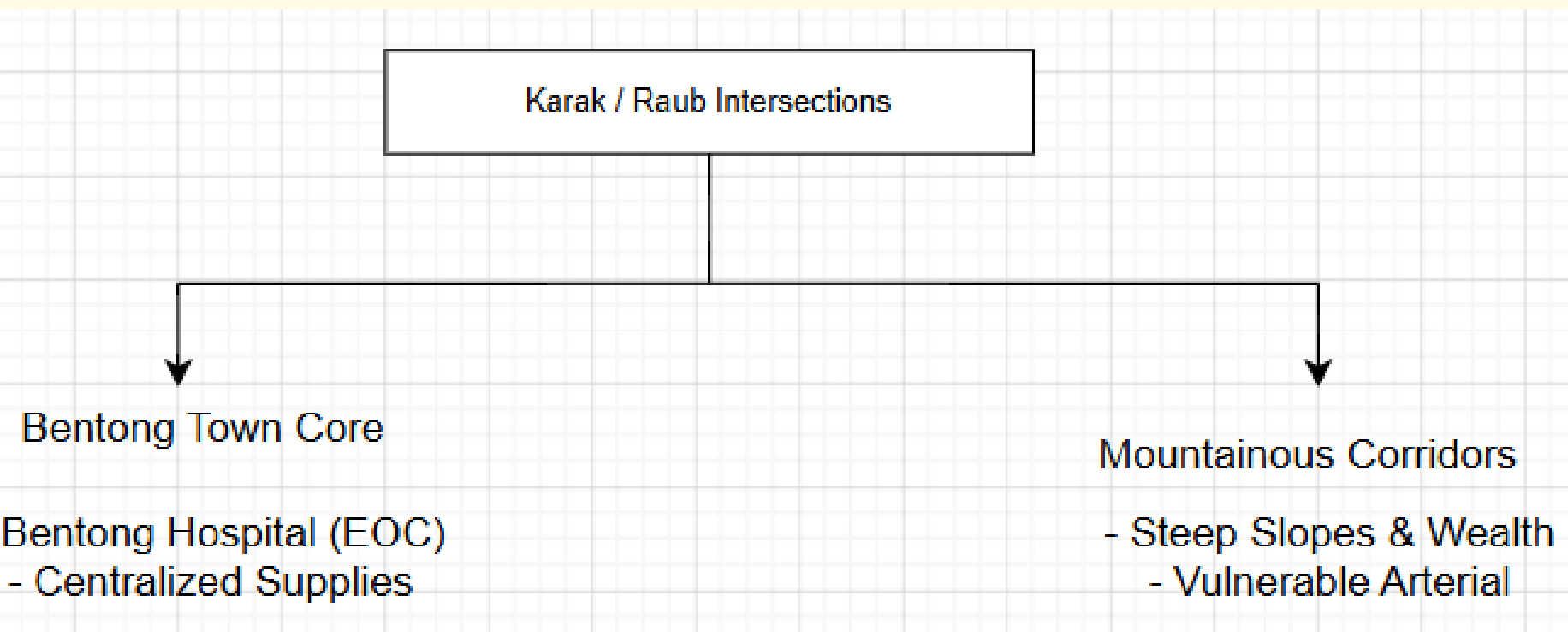
-



- The Operational Leap: Shifting from economic KPIs to pure human survival metrics.
- Cognitive Mitigation: Stripping human error and psychological stress from commanders under crisis.
- Asymptotic Scalability: Sub-exponential design ($O(E + V \log V)$) prevents software lock-ups if chaos expands to Raub or Temerloh.

THE DISASTER RESPONSER CYCLE	
Suboptimal Solution	Optimal AAD Solution
• Guesswork & Backtracking • Wasted Fuel & Vehicles • Bottleneck at EOC	• Instant Obstacle Bypass • Maximized Fleet Lifespan • Synchronized Deliveries
Compound Casualties	Lives Saved ("Golden Hour")

Geographical Network Specification



- The Topology Base: Source Node V0 (Bentong Hospital EOC) to 3 Junction Intersections and 4 isolated target areas.
- Edge Cascades: Multi-wave structural adjustments; routes validated at 08:00 AM can fail by 10:00 AM due to continuing monsoonal rain.

From (Vertex u)	To (Vertex v)	Default Weight (Mins)	Current State	Active Weight in System
Bentong Hospital	Junction B	10	Open	10
Bentong Hospital	Junction A	12	Open	12
Junction B	Village 1	8	Open	8
Junction B	Village 2	5	Open	5
Junction A	Junction C	6	Open	6
Junction C	Village 2	15	Open	15
Junction B	Village 3	9	Blocked	∞ (Purged)
Junction C	Village 3	11	Blocked	∞ (Purged)
Junction C	Village 4	14	Open	14

- Algorithmic Mandate: Single-Source Shortest Path extraction with active edge pruning.
- Definitive Unreachability Flagging: Translating mathematical infinity ($w = \infty$) into life-saving logistical actions (Air-drop escalation for Village 3).

```
=====
BENTONG DISASTER RESPONSE ROUTING ENGINE - OUTPUT LOG
=====
[SOURCE NODE]: Bentong Hospital (EOC)

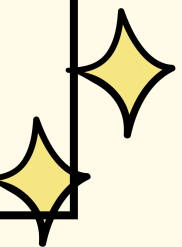
STATUS: SUCCESSFUL PATH COMPUTATION
-----
▶ Village 1: Bentong Hospital → Junction B → Village 1
  Total Estimated Travel Time: 18 Minutes
  Route Status           : CLEAR / OPTIMAL

▶ Village 2: Bentong Hospital → Junction B → Village 2
  Total Estimated Travel Time: 15 Minutes
  Route Status           : CLEAR / ALTERNATIVE DETOUR

▶ Village 3: No available route.
  Total Estimated Travel Time: INF (∞)
  Route Status           : CRITICAL CRASH - VILLAGE IS ISOLATED
  Action Required        : ALERT AIR DISPATCH (HELICOPTER REQ)
=====
```

Algorithm Selection and Justification

Algorithm	Main purpose	Strengths	Weaknesses	Suitability
Sorting	Arrange data	Fast organization, improves search	Cannot find routes	Low
Divide and Conquer	Break problem into smaller parts	Efficient, supports recursion	Not suitable for connected networks	Low
Dynamic Programming	Optimization	Optimal solutions, avoids repeats	High memory, complex	Medium
Greedy	Local optimal decisions	Fast, simple	Not always globally optimal	High
Graph Algorithms	Network/path analysis	Best for routing, accurate model	Requires graph structure	Very High





Algorithm Selection and Justification



Selected algorithm : Dijkstra's algorithm
(graph algorithm that uses a greedy technique)

- ✓ Matches Single-Source Shortest Path (SSSP) problem
- ✓ Works on weighted graphs (distance, time)
- ✓ Efficient and widely used in real-world systems

Advantages of Dijkstra's Algorithm

1. Guaranteed Shortest Path
 - Finds the shortest route to every reachable village
2. Fast Decision Making
 - Enables quick identification of optimal routes
3. Handles Road Closures
 - Easily excludes blocked roads from the network
4. Scalable Solution
 - Adapts to additional villages and roads without redesign
5. Real-World Application
 - Widely used in GPS, transportation, and emergency systems

Scenario Mapping

- Rescue HQ → Source Node
- Villages → Destination Nodes
- Roads → Edges
- Distance → Edge Weight
- Blocked Roads → Removed Edges

Pseudocode

- initialization: Distances = ∞ , Parents = NIL, Source = 0.
- Main Process: Priority Queue $\rightarrow$ Extract-Min $\rightarrow$ Update Neighbors.
- Decision Output:
 - If unreachable $\rightarrow$ "Village isolated – dispatch alert"
 - Else $\rightarrow$ Show path, distance, route status.
- Path Reconstruction: Trace back using parent pointers.

```
1. FOR each vertex v in G:
2.   dist[v]  $\leftarrow \infty$ 
3.   parent[v]  $\leftarrow$  NIL
4.   visited[v]  $\leftarrow$  FALSE
5. END FOR

6. dist[source]  $\leftarrow$  0
7. Insert all vertices into Priority Queue Q

8. WHILE Q is not empty:
9.   u  $\leftarrow$  EXTRACT-MIN(Q)           // vertex with smallest dist[]
10.  visited[u]  $\leftarrow$  TRUE

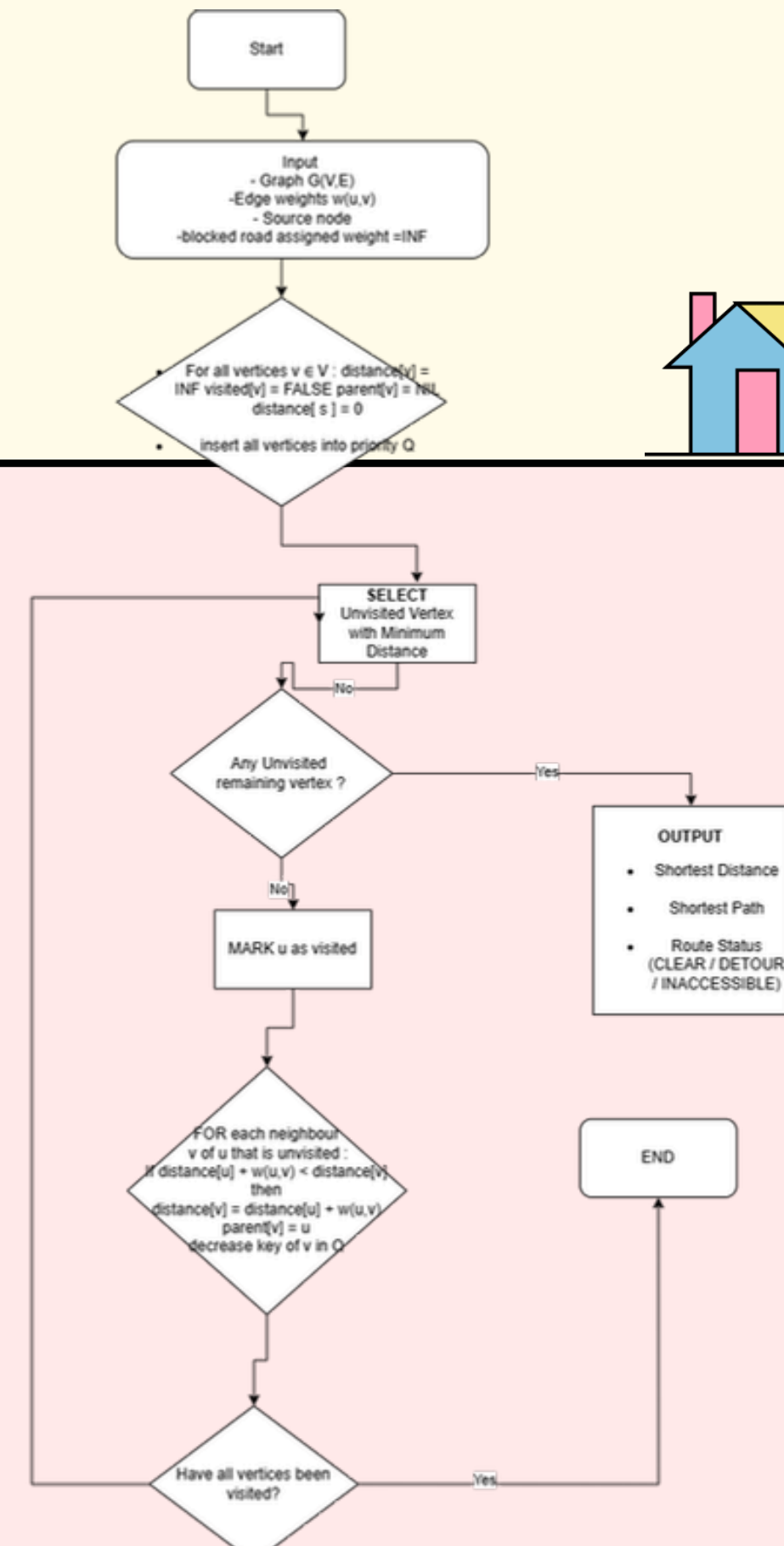
11.  FOR each neighbour v of u:
12.    IF visited[v] = FALSE AND w(u,v)  $\neq \infty$ :
13.      IF dist[u] + w(u,v) < dist[v]:
14.        dist[v]  $\leftarrow$  dist[u] + w(u,v)
15.        parent[v]  $\leftarrow$  u
16.        UPDATE Q with new dist[v]
17.      END IF
18.    END IF
19.  END FOR
20. END WHILE

21. FOR each destination village d:
22.  IF dist[d] =  $\infty$ :
23.    OUTPUT "VILLAGE IS ISOLATED – ALERT AIR DISPATCH"
24.  ELSE:
25.    OUTPUT reconstructPath(d), dist[d], routeStatus(d)
26.  END IF
27. END FOR

RECONSTRUCT-PATH(d):
  path  $\leftarrow$  empty list
  WHILE d  $\neq$  NIL:
    prepend d to path
    d  $\leftarrow$  parent[d]
  RETURN path
```

Flowchart

- Road network modeled as weighted graph; blocked roads set to ∞ .
- Initialize distances to ∞ , parents NIL, source (Bentong Hospital) = 0.
- Use priority queue to pick nearest unvisited node.
- Relax neighbors: update distance and parent if shorter path found.
- Continue until all reachable nodes are processed.
- Output shortest distance, reconstructed path, and route status (CLEAR / DETOUR / INACCESSIBLE).



Technical Implementation: Landslide Simulation & Dijkstra Loop

- Interconnected Architecture: System directly bridges environmental simulation (`blockRoad`) with greedy path execution (`while loop`)
- **Step 1:** Simulating the Landslide (`blockRoad`): Field reports call `blockRoad (source, dest)` dynamically
 - Toggles the targeted edge's boolean flag `isBlocked` from `false` to `true`
- **Step 2:** Core Dijkstra Execution Loop (`while`): Permanently extracts the closest unvisited node `u` from the priority queue
 - Iterates through all adjacent neighbors `v` to find valid onward paths
- The Intersection of Logic (`if` Statement): Loop evaluates `if (!edge.isBlocked)` before doing anything else
 - if landslide blocked the road, algorithm instantly skips that path
 - this prevents relaxation step (`dist[u] + weight < dist[v]`), forcing a safe detour or declaring a village isolated (`INF`)

```
public void blockRoad(int u, int v) {
    for (Edge edge : adjList.get(u)) {
        if (edge.targetNode == v) edge.isBlocked = true;
    }
    for (Edge edge : adjList.get(v)) {
        if (edge.targetNode == u) edge.isBlocked = true;
    }
}
```

```
while (!pq.isEmpty()) {
    PQNode current = pq.poll();
    int u = current.nodeId;

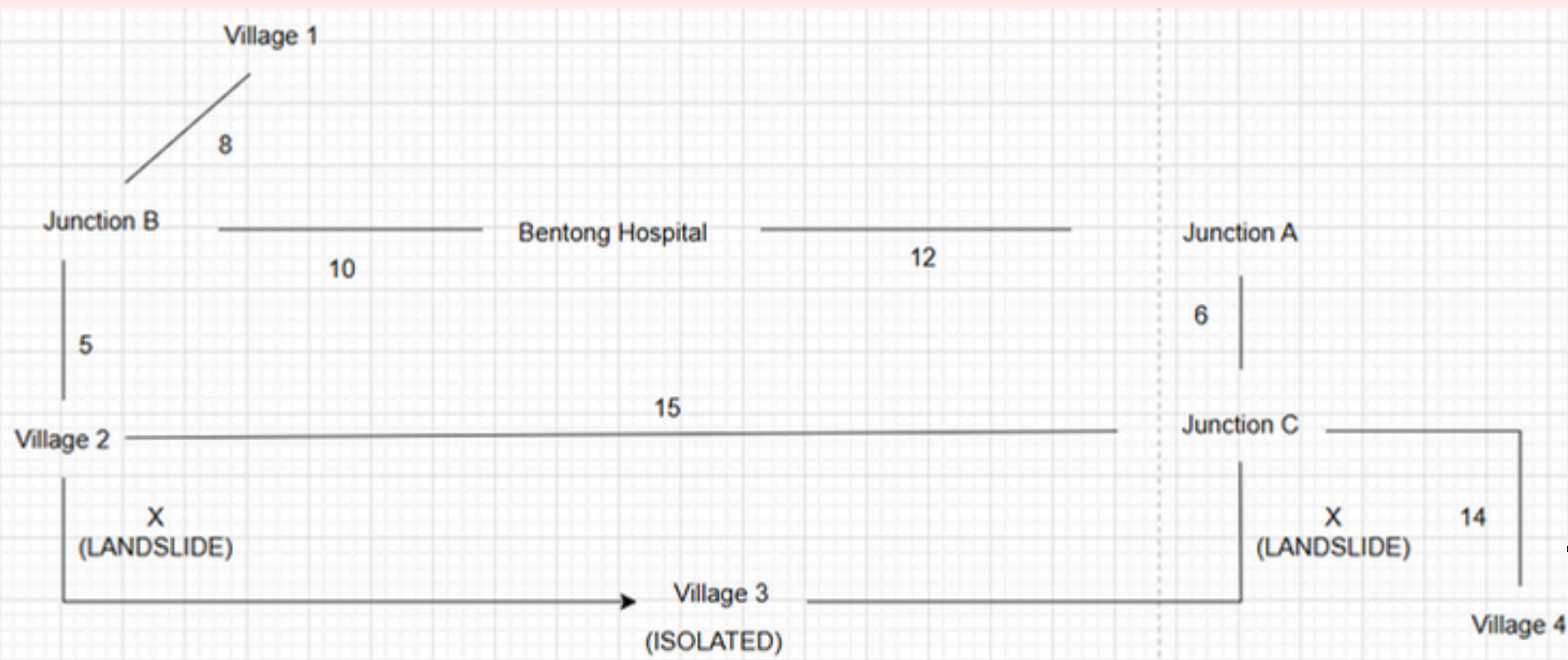
    if (current.accumulatedTime > distances[u]) continue;

    for (Edge edge : graph.adjList.get(u)) {
        // Flowchart Check: "Is the edge blocked by a landslide?"
        if (edge.isBlocked) continue;

        int v = edge.targetNode;
        int weight = edge.travelTimeMinutes;

        // Flowchart Check: "distance[u] + weight < distance[v]"
        if (distances[u] + weight < distances[v]) {
            distances[v] = distances[u] + weight;
            parent[v] = u;
            pq.add(new PQNode(v, distances[v]));
        }
    }
}
```

Program Output



```
PS C:\Users\User> & 'C:\Program Files\Microsoft\jdk-21.0.11.10-hotspot\bin\j
odesws_9ac87\jdt_ws\jdt.ls-java-project\bin' 'LandslideEmergencyRouting'
=====
BENTONG DISASTER RESPONSE ROUTING ENGINE - OUTPUT LOG
=====
[SOURCE NODE]: Bentong Hospital (EOC)
STATUS: SUCCESSFUL PATH COMPUTATION

* Village 1: Bentong Hospital (EOC) -> Junction B -> Village 1
  Total Estimated Travel Time: 18 Minutes
  Route Status                  : CLEAR / OPTIMAL

* Village 2: Bentong Hospital (EOC) -> Junction B -> Village 2
  Total Estimated Travel Time: 15 Minutes
  Route Status                  : CLEAR / ALTERNATIVE DETOUR

* Village 3: No available route.
  Total Estimated Travel Time: INF (∞)
  Route Status                  : CRITICAL CRASH - VILLAGE IS ISOLATED
  Action Required               : ALERT AIR DISPATCH (HELICOPTER REQ)

* Village 4: Bentong Hospital (EOC) -> Junction A -> Junction C -> Village 4
  Total Estimated Travel Time: 32 Minutes
  Route Status                  : CLEAR / ALTERNATIVE DETOUR
=====
```

Output Breakdown:

- Village 1: **EOC → Junction B → Village 1** (18 Minutes)
- Village 2: **EOC → Junction B → Village 2** (15 Minutes)
- Village 3: **INF distance** & triggered critical warning message
- Village 4: **EOC → Junction A → Junction C → Village 4** (32 Minutes)





Correctness Proof

Why Dijkstra's Algorithm Produces Correct Results

- Source node (Bentong Hospital) is initialised with distance = 0
- All other vertices are initialised with distance = ∞
- The algorithm repeatedly selects the unvisited vertex with the smallest tentative distance
- Edge relaxation updates shorter paths when found
- All road weights are non-negative
- Once a vertex is marked as visited, its shortest path is finalised

Time Complexity Analysis

Complexity Type	Result
Best Case	$O((V + E) \log V)$
Average Case	$O((V + E) \log V)$
Worst Case	$O((V+E)\log V) \rightarrow O(V^2 \log V)$ if $E \approx V^2$

Where:

V = Number of vertices

E = Number of edges

Min Priority Queue is used

Why?

Each vertex is processed

Each edge may be relaxed

Heap operations require $O(\log V)$

Asymptotic Analysis

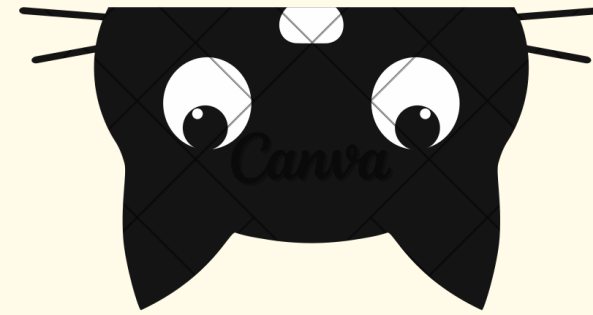
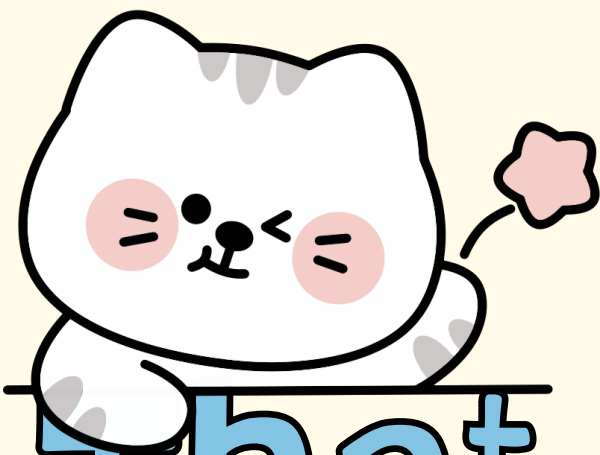
Complexity Type	Result
Best Case	$O((V + E) \log V)$
Average Case	$O((V + E) \log V)$
Worst Case	$O((V+E)\log V) \rightarrow O(V^2 \log V)$ if $E \approx V^2$

Space Complexity

$$O(V+E)$$

Additional Memory Used

- Adjacency List
- distance[]
- visited[]
- parent[]
- Priority Queue



That is all from us...

Thank You For
Your Attention!

